

Metodología I

Unidad 7 Testing

Nombre: Farias, Gustavo

Comisión: M2025-13

Matrícula: 101662

Repositorio GitHub:

https://github.com/Lucenear/Tecnicatura_Programacion_UTN/tree/main/3er_Semestre/Metodologia_I

Parte 1: Fundamentos y Mentalidad del Tester

1. Testing de Software y el Costo Exponencial

El Testing de Software es un proceso sistemático de evaluación de un sistema o componente mediante ejecución manual o automática, con el objetivo de verificar que cumple con los requisitos especificados y detectar defectos antes de que lleguen al usuario final. No se trata solo de "buscar errores", sino de validar que el software aporta valor y funciona según lo esperado en condiciones reales.

La metáfora del "Costo Exponencial de un Defecto" ilustra cómo el precio de corregir un bug aumenta drásticamente cuanto más tarde se detecta. Un error identificado en la fase de diseño puede costar \$1; si se encuentra durante el desarrollo, cuesta \$10; si escapa a pruebas de aceptación, \$100; y si llega a producción, puede costar \$1000 o más (incluyendo parches urgentes, pérdida de reputación, soporte técnico y posibles multas).

Por esto, el testing se considera una inversión y no un gasto: prevenir defectos tempranamente ahorra dinero, tiempo y recursos a largo plazo, protegiendo la calidad del producto y la satisfacción del cliente.

2. La Mentalidad del Tester (Ejemplo: Formulario de Login)

Un tester posee una mentalidad crítica, curiosa y orientada a explorar escenarios donde el sistema podría fallar, a diferencia del desarrollador que suele enfocarse en el "camino feliz". Ante un formulario de login, un tester preguntaría:

- Qué pasa si ingreso una contraseña válida pero con espacios en blanco al inicio o al final? (El sistema los trimmea automáticamente o rechaza el acceso?)
- Qué sucede si intento iniciar sesión 10 veces seguidas con contraseñas incorrectas en menos de un minuto? (Existe protección contra fuerza bruta o bloqueo temporal de cuenta?)
- Qué ocurre si pego una cadena de 5000 caracteres en el campo de usuario o inyecto código SQL (' OR 1=1 --)? (El sistema valida la entrada o es vulnerable a ataques?)

3. Contraste de Enfoques: Caja Negra, Blanca y Gris

Enfoque	Objetivo Principal	Ejemplo Exclusivo
Caja Negra	Validar la funcionalidad externa sin conocer la estructura interna. Se basa en requisitos y especificaciones.	Probar que el botón "Pagar" redirige correctamente a la pasarela de pago ingresando datos válidos, sin importar cómo esté codificado el backend.

Caja Blanca	Verificar la lógica interna, estructuras de control y flujo de código. Requiere acceso al código fuente.	Analizar si una condición `IF-ELSE` anidada en el cálculo de impuestos cubre todos los caminos lógicos posibles mediante cobertura de sentencias.
Caja Gris	Combinar ambos enfoques: probar la funcionalidad externa teniendo conocimiento parcial de la arquitectura interna para diseñar casos más efectivos.	Diseñar pruebas de API sabiendo que el endpoint usa caché Redis; se prueba la respuesta correcta (negra) pero también se verifica que la caché se invalide tras una actualización (blanca).

Parte 2: Técnicas de Diseño de Pruebas de Caja Negra

Requisito 1: Campo "Edad" (18 a 65 años inclusive)

Particiones de Equivalencia (5 clases)

- 1- Válida: Edad entre 18 y 65 (ej: 30).
- 2- Inválida (Menor): Edad menor a 18 (ej: 17).
- 3- Inválida (Mayor): Edad mayor a 65 (ej: 66).
- 4- Inválida (No numérico): Caracteres alfabéticos o símbolos (ej: "veinte").
- 5- Inválida (Vacío/Nulo): Campo sin ingresar ningún valor.

Análisis de Valores Límite (BVA) - 6 Casos de Prueba

- 1- 17 (Límite inferior inválido - 1)
- 2- 18 (Límite inferior válido)
- 3- 19 (Límite inferior válido + 1)
- 4- 64 (Límite superior válido - 1)
- 5- 65 (Límite superior válido)
- 6- 66 (Límite superior inválido + 1)

Requisito 2: Regla de Negocio (Descuento 15% si Premium Y Compra > \$50.000)

Tabla de Decisión

Condición / Regla	R1	R2	R3	R4
Cliente es Premium	SI	SI	NO	NO
Compra > \$50.000	SI	NO	SI	NO
Acción: Aplicar 15% Descuento	SI	NO	NO	NO

Por qué probar solo la Regla 1 es insuficiente?

Probar únicamente el caso "Premium Y Compra > \$50.000" (R1) valida que el descuento se aplica, pero deja completamente expuestos tres escenarios críticos donde el descuento no debe aplicarse. Sin probar R2, R3 y R4, no podemos garantizar que el sistema no esté otorgando beneficios erróneamente a clientes no premium o con compras menores, lo cual generaría pérdidas económicas directas para el negocio. La tabla de decisión asegura que todas las combinaciones lógicas sean verificadas.

Parte 3: Pruebas Estructurales y Niveles de Prueba

1. Cobertura de Código (Code Coverage)

Es una métrica cuantitativa que indica qué porcentaje del código fuente ha sido ejecutado durante las pruebas. No garantiza que el software esté libre de bugs, pero sí señala áreas no probadas que representan riesgos potenciales.

2. Análisis del Pseudocódigo es_mayor_de_edad

Funcion es_mayor_de_edad(edad):

if edad >= 18:

Imprimir("Es mayor")

else:

Imprimir("Es menor")

;

- Insuficiencia de la Prueba 1 (edad = 25): Aunque la prueba funciona y muestra "Es mayor", solo recorre la rama TRUE del condicional. La rama ELSE (cuando edad < 18) permanece totalmente inexplorada. Si hubiera un error en esa rama (ej: imprimir "Es mayor" por error de copia-pegar), esta prueba nunca lo detectaría.
- Porcentaje de Cobertura de Decisión: Con solo edad=25, se alcanza el 50% de cobertura de decisión (1 de 2 ramas ejecutadas).
- Casos para 100% de Cobertura de Decisión: Se necesitan dos valores:
 - edad = 25 (o cualquier valor ≥ 18) → Cubre rama TRUE.
 - edad = 10 (o cualquier valor < 18) → Cubre rama FALSE.

3. Pirámide del Testing

La Pirámide del Testing es un modelo visual que recomienda distribuir las pruebas en proporciones equilibradas para maximizar eficiencia y minimizar costos de mantenimiento:

- Base (Pruebas Unitarias): Son la mayor cantidad. Prueban funciones/métodos aislados. Son rápidas, baratas y fáciles de mantener. Detectan bugs temprano.
- Centro (Pruebas de Integración): Verifican la interacción entre módulos, APIs o bases de datos. Son menos numerosas porque son más lentas y complejas.

- Cúspide (Pruebas de Sistema/E2E): Simulan flujos completos de usuario. Son las menos numerosas porque son lentas, frágiles y costosas de mantener. Sirven como validación final de negocio.

4. Pruebas Funcionales vs. No Funcionales

- Funcionales: Validan qué hace el sistema respecto a los requisitos de negocio.
 - Ejemplo: Verificar que al hacer clic en "Agregar al carrito", el producto se suma correctamente al total.
- No Funcionales: Validan cómo se comporta el sistema bajo ciertas condiciones (rendimiento, seguridad, usabilidad).
 - Ejemplo: Medir que la página de checkout cargue en menos de 2 segundos con 1000 usuarios concurrentes.

Parte 4: Aplicaciones en el Mundo Real e Investigación

1. Falla Famosa: El Vuelo 501 de Ariane 5 (1996)

El cohete Ariane 5 explotó 37 segundos después del despegue debido a un error de software: un valor de velocidad horizontal de 64 bits fue convertido incorrectamente a un entero de 16 bits, causando un desbordamiento (overflow). Este bug existía en el código heredado del Ariane 4, pero las condiciones de vuelo del Ariane 5 eran distintas y activaron el error.

Nivel de prueba fallido: Pruebas de Integración y Sistema

Justificación: Las pruebas unitarias del módulo de conversión probablemente pasaron porque el código funcionaba bien dentro del rango del Ariane 4. El fallo ocurrió porque no se integró ni probó ese módulo específico bajo las nuevas condiciones operativas del Ariane 5. Fue una falla de validación de sistema completo ante un cambio de entorno, no un error de lógica aislada.

2. Preguntas de Investigación

Pregunta 1: Selenium vs. Cypress (Enfoque Práctico)

Aspecto	Selenium	Cypress
Arquitectura	Se comunica con el navegador mediante drivers externos (WebDriver). Ejecuta comandos fuera del browser.	Se ejecuta *dentro* del mismo ciclo de ejecución del navegador (mismo loop de eventos). Control directo del DOM.
Lenguajes	Java, Python, C#, JavaScript, Ruby. Multi-lenguaje.	Solo JavaScript / TypeScript.

Velocidad y Estabilidad	Más lento por la comunicación externa. Propenso a flakes (pruebas intermitentes) por sincronización.	Más rápido y estable. Espera automática de elementos, sin necesidad de sleeps explícitos.
Cuándo usar Selenium	Cuando necesitas probar en múltiples navegadores (IE, Safari legacy), trabajar con equipos que usan Java/C#, o automatizar flujos cross-browser complejos.	Ideal para aplicaciones web modernas (React, Vue, Angular), equipos frontend JS/TS, y cuando se prioriza velocidad de feedback en CI/CD.

Fuente:

- Blog de QA Testing Latinoamérica. "Selenium vs Cypress: ¿Cuál elegir en 2024?". Disponible en: gatestinglatam.com

Pregunta 2: Testing de Regresión en Sprints y CI/CD

El testing de regresión consiste en re-ejecutar pruebas existentes para asegurar que nuevos cambios (features, fixes, refactorings) no han roto funcionalidades que ya funcionaban correctamente.

¿Por qué es ideal para automatización en metodologías ágiles?

En un enfoque ágil, los Sprints duran entre 1 y 4 semanas y entregan incrementos constantes. Hacer regresión manual en cada Sprint es imposible: el suite de pruebas crece exponencialmente y el tiempo de sprint es limitado. La automatización permite:

- Integración Continua (CI): Cada vez que un desarrollador hace push de código, el pipeline de CI ejecuta automáticamente el suite de regresión. Si algo se rompió, el equipo lo sabe en minutos, no días.
- Feedback Inmediato: Los testers pueden enfocarse en pruebas exploratorias y de nuevo valor, mientras la máquina valida que lo existente sigue funcionando.
- Confianza para Desplegar: En CD (Continuous Delivery), la regresión automatizada actúa como red de seguridad que habilita despliegues frecuentes sin miedo a introducir bugs inadvertidos.

Sin automatización de regresión, los equipos ágiles terminan acumulando deuda técnica de calidad o reduciendo la cobertura de pruebas por falta de tiempo, contradiciendo el principio ágil de "software funcionando sobre documentación extensiva".

Fuente:

- Comunidad Agile Argentina. "Automatización de pruebas en Scrum: Guía práctica". agileargentina.org.ar